大模型问答-开放服务接口

大模型问答依托安全垂域大模型，秉承让安全更智能、让智能更安全的使命，基于底层多源异构模型、算法调度引擎及海量安全知识，具备恶意代码检测、威胁情报分析、自动化安全编排响应和安全教育等能力，可根据各业务场景需求从容切换多种安全角色，以扎实的安全基本功迎接未来无限可能。

开放接口

项	内容
文档版本	v1.5
更新日期	2026-07-21
兼容标准	OpenAI Chat Completions API

---

1. 概述

本平台提供 **OpenAI 兼容** 的大模型对话接口，调用方使用平台分配的 **LLMKey** 进行鉴权，请求经能力中心网关转发至底层模型服务。

适用场景：业务系统、脚本或第三方应用以 HTTP API 方式调用平台大模型能力。

---

2. 接入前置条件

- 租户已开通大模型接口访问
- 用户已分配 LLMKey
- 用户账号有效（未禁用、未过期）
- 租户账号有效（未禁用）

---

3. 鉴权方式

所有对外大模型接口均使用 **HTTP Bearer Token** 鉴权，将 LLMKey 放入请求头：

```
Authorization: Bearer <LLMKey>
Content-Type: application/json
```

说明：

- 仅支持 `Authorization: Bearer <token>` 格式，不支持 Query 参数传 Key
- 本接口**不需要**平台 Cookie / JWT 登录态
- LLMKey 与用户绑定；同一租户下不同用户使用各自的 LLMKey

---

4. 接口列表

方法	路径	说明
POST	`/api/openai/v1/chat/completions`	对话补全（支持流式 / 非流式）

完整 URL 示例：

```
https://{host}/api/openai/v1/chat/completions
```

其中 `{host}` 为能力中心服务地址（与平台 Web 访问域名一致）。

---

5. 对话补全

5.1 请求

POST `/api/openai/v1/chat/completions`

请求头

名称	必填	说明
`Authorization`	是	`Bearer <LLMKey>`
`Content-Type`	是	`application/json`

请求体

字段	类型	必填	说明
`model`	String	是	模型名称，如 `HengNao-v4`（须为平台已接入且租户可用的模型）
`messages`	Array	是	对话消息列表，消息字段见下表
`temperature`	Number	否	采样温度，参考范围 `0-2`
`top_p`	Number	否	核采样参数，参考范围 `0-1`
`n`	Integer	否	生成结果数量
`stream`	Boolean	否	是否流式输出，默认 `false`
`stream_options`	Object	否	流式响应选项，例如 `{"include_usage": true}`
`stop`	String | Array<String>	否	停止序列，支持单个字符串或字符串数组
`max_tokens`	Integer	否	最大生成 Token 数
`max_completion_tokens`	Integer	否	最大补全 Token 数
`presence_penalty`	Number	否	主题重复惩罚，参考范围 `-2` 至 `2`
`frequency_penalty`	Number	否	词频重复惩罚，参考范围 `-2` 至 `2`
`logit_bias`	Object	否	Token ID 到偏置值的映射
`user`	String | Object	否	调用方标识或模型服务支持的用户信息
`tools`	Array	否	可调用工具定义，见“工具调用”示例
`tool_choice`	String | Object	否	工具选择策略或指定工具
`response_format`	Object	否	响应格式，如 `text`、`json_object`、`json_schema`
`structured_outputs`	Object	否	结构化输出约束，支持 `choice`、`json`、`regex`、`grammar`、`structural_tag` 等模型能力
`chat_template_kwargs`	Object	否	Chat Template 扩展参数，例如 `{"enable_thinking": true}`
`seed`	Integer	否	随机种子
`reasoning_effort`	String	否	推理强度：`low`、`medium`、`high`
`modalities`	Array<String>	否	输出模态列表，例如 `["text"]` 或 `["text", "audio"]`

字段	类型	必填	说明
`audio`	Object	否	音频输出配置，常用字段为 `voice`、`format`

消息字段：

字段	类型	必填	说明
`messages[].role`	String	是	`system`、`user`、`assistant`、`tool` 或 `developer`
`messages[].content`	String | Array | null	按角色	文本或 OpenAI Vision 内容块数组；仅发起工具调用时可为 `null`
`messages[].name`	String	否	消息发送者名称
`messages[].tool_calls`	Array	否	assistant 发起的工具调用列表
`messages[].tool_calls[].id`	String	否	工具调用 ID
`messages[].tool_calls[].type`	String	否	工具类型，当前通常为 `function`
`messages[].tool_calls[].function`	Object	否	函数调用信息
`messages[].tool_calls[].function.name`	String	否	函数名称
`messages[].tool_calls[].function.arguments`	String	否	函数参数，内容为 JSON 字符串
`messages[].tool_call_id`	String	否	tool 角色返回结果时对应的工具调用 ID
`messages[].reasoning_content`	String	否	推理内容

嵌套对象字段：

字段	类型	必填	说明
`stream_options.include_usage`	Boolean	否	流式响应是否包含 Token 用量分片
`tools[].type`	String	否	工具类型，当前通常为 `function`
`tools[].function`	Object	否	函数定义
`tools[].function.name`	String	否	函数名称
`tools[].function.description`	String	否	函数用途说明
`tools[].function.parameters`	Object	否	JSON Schema 格式的函数参数定义
`response_format.type`	String	否	`text`、`json_object` 或 `json_schema`
`response_format.json_schema`	Object	否	`json_schema` 类型使用的 JSON Schema 定义
`audio.voice`	String	否	音频输出使用的声音
`audio.format`	String	否	音频输出格式

> 上述字段均由网关透传至模型服务。参数是否生效取决于目标模型的实际能力，网关不替目标模型补齐默认值。
>
> `messages[].content` **数组格式（Vision）**：当 `content` 为数组时，每个元素为一个内容块，支持 `text` 和 `image_url` 两种类型：
>
> ```
> "content": [
>   {"type": "text", "text": "请描述这张图片"},
>   {"type": "image_url", "image_url": {"url": "...",
> "detail": "auto"}}
> ]
> ```
>
> **图片 URL 限制**：`image_url.url` 仅支持纯 Base64 字符串或 Base64 Data URI（`...`），不支持 `http://` 或 `https://` 链接。若传入非 Base64 的图片 URL，请求将被拒绝并返回参数校验错误。

请求示例（非流式）

```
curl -sS -X POST "https://{host}/api/openai/v1/chat/completions" \
  -H "Authorization: Bearer e927b642dfbe4604b1388ec0732aa45a.wBW4eUMF8FZ4zDmJ" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "HengNao-v4",
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": "Hello"}
    ],
    "temperature": 0.7,
```

```
    "max_tokens": 2048,
    "stream": false
  }'
```

请求示例（流式）

```
curl -sS -N -X POST "https://{host}/api/openai/v1/chat/completions" \
  -H "Authorization: Bearer e927b642dfbe4604b1388ec0732aa45a.wBW4eUMF8FZ4zDmJ" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "HengNao-v4",
    "messages": [
      {"role": "user", "content": "用三句话介绍恒脑"}
    ],
    "stream": true
  }'
```

请求示例（图片识别 / Vision）

```
curl -sS -X POST "https://{host}/api/openai/v1/chat/completions" \
  -H "Authorization: Bearer e927b642dfbe4604b1388ec0732aa45a.wBW4eUMF8FZ4zDmJ" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "HengNao-v4",
    "messages": [
      {
        "role": "user",
        "content": [
          {"type": "text", "text": "请描述这张图片"},
          {"type": "image_url", "image_url": {"url": "...",
"detail": "auto"}}
        ]
      }
    ],
    "stream": false
  }'
```

请求示例（工具调用）

`tool_choice` 可传 `auto`、`required`、`none`，也可使用对象指定函数。

```
curl -sS -X POST "https://{host}/api/openai/v1/chat/completions" \
  -H "Authorization: Bearer <LLMKey>" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "HengNao-r1",
    "messages": [
```

```
      {"role": "user", "content": "杭州天气怎么样？"}
    ],
    "tools": [
      {
        "type": "function",
        "function": {
          "name": "get_weather",
          "description": "查询天气",
          "parameters": {
            "type": "object",
            "properties": {
              "city": {"type": "string"}
            },
            "required": ["city"]
          }
        }
      }
    ],
    "tool_choice": "auto"
  }'
```

模型返回 `tool_calls` 后，调用方执行工具并通过 `tool` 角色回传结果：

```
{
  "model": "HengNao-r1",
  "messages": [
    {
      "role": "assistant",
      "content": null,
      "tool_calls": [
        {
          "id": "call-1",
          "type": "function",
          "function": {
            "name": "get_weather",
            "arguments": "{\"city\":\"杭州\"}"
          }
        }
      ]
    },
    {
      "role": "tool",
      "tool_call_id": "call-1",
      "content": "晴，30°C"
    }
  ]
}
```

请求示例（结构化输出与推理）

```
curl -sS -X POST "https://{host}/api/openai/v1/chat/completions" \
  -H "Authorization: Bearer <LLMKey>" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "HengNao-r1",
    "messages": [
      {"role": "user", "content": "生成一个人的姓名和年龄"}
    ],
    "response_format": {
      "type": "json_schema",
      "json_schema": {
        "name": "person",
        "schema": {
          "type": "object",
          "properties": {
            "name": {"type": "string"},
            "age": {"type": "integer"}
          },
          "required": ["name", "age"]
        }
      }
    },
    "chat_template_kwargs": {
      "enable_thinking": true
    },
    "reasoning_effort": "high"
  }'
```

5.2 响应

成功响应（非流式）

HTTP 状态码：`200`

响应体为 **OpenAI 标准 JSON**。

```
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "created": 1749123456,
  "model": "HengNao-v4",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I help you?"
```

```
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 8,
    "total_tokens": 18
  }
}
```

非流式响应顶层字段：

字段	类型	说明
`id`	String	本次补全请求的唯一标识
`object`	String	对象类型，非流式通常为 `chat.completion`
`created`	Integer	响应创建时间，Unix 时间戳（秒）
`model`	String	实际生成响应的模型名称
`choices`	Array	生成结果列表
`usage`	Object	Token 用量；模型服务不返回统计时可能缺省
`system_fingerprint`	String	模型服务系统指纹；仅部分模型返回

`choices` 与 `message` 字段：

字段	类型	说明
`choices[].index`	Integer	当前结果在 `choices` 中的序号
`choices[].message`	Object	当前生成的完整消息
`choices[].message.role`	String	响应消息角色，通常为 `assistant`
`choices[].message.content`	String | Array | null	模型生成的文本或多模态内容；发起工具调用时可能为 `null`
`choices[].message.name`	String	消息发送者名称；仅部分模型返回
`choices[].message.tool_calls`	Array	模型请求调用的工具列表
`choices[].message.tool_calls[].id`	String	工具调用 ID，回传工具结果时用于 `tool_call_id`
`choices[].message.tool_calls[].type`	String	工具类型，当前通常为 `function`
`choices[].message.tool_calls[].function`	Object	函数调用信息
`choices[].message.tool_calls[].function.name`	String	函数名称
`choices[].message.tool_calls[].function.arguments`	String	函数参数，内容为 JSON 字符串，调用方需解析后使用
`choices[].message.tool_call_id`	String	与工具调用关联的 ID；仅部分响应类型返回
`choices[].message.reasoning`	String	推理内容；模型扩展字段，仅部分推理模型返回
`choices[].message.reasoning_content`	String	推理内容；模型扩展字段，仅部分推理模型返回
`choices[].message.audio`	Object	音频输出；仅请求音频模态且模型支持时返回，结构以目标模型为准
`choices[].finish_reason`	String	结束原因：`stop`、`length`、`tool_calls` 或 `content_filter`

`usage` 字段：

字段	类型	说明
`usage.prompt_tokens`	Integer	输入 Token 数
`usage.completion_tokens`	Integer	输出 Token 数
`usage.total_tokens`	Integer	总 Token 数
`usage.prompt_tokens_details`	Object	输入 Token 分类明细；仅部分模型返回
`usage.prompt_tokens_details.cached_tokens`	Integer	缓存命中的输入 Token 数
`usage.prompt_tokens_details.text_tokens`	Integer	文本输入 Token 数
`usage.prompt_tokens_details.audio_tokens`	Integer	音频输入 Token 数
`usage.prompt_tokens_details.image_tokens`	Integer	图片输入 Token 数
`usage.completion_tokens_details`	Object	输出 Token 分类明细；仅部分模型返回
`usage.completion_tokens_details.text_tokens`	Integer	文本输出 Token 数
`usage.completion_tokens_details.audio_tokens`	Integer	音频输出 Token 数
`usage.completion_tokens_details.reasoning_tokens`	Integer	推理 Token 数

成功响应（流式 SSE）

HTTP 状态码：`200`

`Content-Type` 一般为 `text/event-stream`。响应体为 SSE 事件流，格式与 OpenAI 流式接口一致：

```
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"index":0,"delta":
{"content":"你"},"finish_reason":null}]}

data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"index":0,"delta":
{"content":"好"},"finish_reason":null}]}

data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"index":0,"delta":
{},"finish_reason":"stop"}]}

data: [DONE]
```

流式响应字段：

字段	类型	说明
`id`	String	本次补全请求的唯一标识
`object`	String	对象类型，流式通常为 `chat.completion.chunk`
`created`	Integer	响应创建时间，Unix 时间戳（秒）
`model`	String	实际生成响应的模型名称
`choices[].index`	Integer	当前结果序号
`choices[].delta.role`	String	增量消息角色，通常只在首个分片出现
`choices[].delta.content`	String | null	增量文本内容
`choices[].delta.reasoning`	String	增量推理内容；模型扩展字段
`choices[].delta.reasoning_content`	String	增量推理内容；模型扩展字段
`choices[].delta.tool_calls`	Array	增量工具调用信息
`choices[].delta.tool_calls[].index`	Integer	工具调用在列表中的序号
`choices[].delta.tool_calls[].id`	String	工具调用 ID，通常只在首个相关分片出现
`choices[].delta.tool_calls[].type`	String	工具类型，当前通常为 `function`
`choices[].delta.tool_calls[].function.name`	String	函数名称，可能仅在首个相关分片出现
`choices[].delta.tool_calls[].function.arguments`	String	函数参数的增量片段；需按 `index` 顺序拼接后再解析 JSON
`choices[].finish_reason`	String | null	未结束时为 `null`；结束时为 `stop`、`length`、`tool_calls` 或 `content_filter`
`usage`	Object | null	使用 `stream_options.include_usage=true` 时，模型服务可在结束前返回用量分片
`system_fingerprint`	String	模型服务系统指纹；仅部分模型返回

每个 SSE 事件的 `data:` 后是一个 JSON 分片；`data: [DONE]` 表示流结束，不是 JSON 响应字段。

5.3 错误响应

鉴权失败、权限不足、并发超限等错误时，HTTP 状态码通常为 `200`，响应体为平台统一结构：

```
{
  "code": 401,
  "msg": "未授权",
  "data": null
}
```

code	典型 msg	说明
`401`	Missing or invalid Authorization header	未携带 `Authorization` 头，或格式不正确
`401`	Invalid API key	LLMKey 无效或不存在
`401`	用户已过期，请联系系统管理员	用户已超过到期时间（i18n，文案随 `Accept-Language` 变化）
`401`	账户已被禁用，请联系系统管理员	用户已被禁用（i18n，文案随 `Accept-Language` 变化）
`-28`	OpenAPI access is disabled for this tenant	租户未开通大模型接口访问（`openapiAccess = false`）
`-1`	Rate limit exceeded	超过租户配置的最大并发数
`-1`	Internal server error	服务内部异常
`-1`	must not be blank! / must not be null! / image_url.url must be base64!	参数校验失败（`model` 为空、`messages` 为空或图片 URL 非 Base64）
`-4`	Upstream request failed / 上游错误信息	模型网关调用失败

> `msg` 字段支持国际化，实际返回文案以请求 `Accept-Language` 及平台配置为准。

错误示例

```
{
  "code": -28,
  "msg": "OpenAPI access is disabled for this tenant",
  "data": null
}
```

---

6. 并发与限流

- 并发限制按 **租户维度** 统计，上限由租户字段 `openapiMaxConcurrent` 配置
- 默认值：`10`
- 可配置范围：`1` ~ `500`

- 超出并发上限时返回 `code = -1`，`msg = Rate limit exceeded`

同一租户下所有用户的 LLMKey 共享该并发配额。

---

7. 访问统计

每次成功调用产生的 Token 消耗会记录访问日志，包含租户、用户 ID、客户端 IP、模型名称、Token 用量、耗时等信息。

调用时的客户端 IP 从请求头 `X-Forwarded-For`、`X-Real-IP` 或直连地址解析。

---

8. LLMKey 生命周期

事件	行为
租户开通大模型接口访问后创建用户	自动生成 LLMKey
租户关闭大模型接口访问	LLMKey 鉴权失败；平台「接口认证信息」不再返回 `llmKey`
用户被禁用或过期	LLMKey 鉴权失败（`401`）
用户被逻辑删除	LLMKey 自然失效，鉴权查询无法命中

---

9. 调用建议

1. **妥善保管 LLMKey**：等同于 API 密钥，勿写入前端代码或公开仓库
2. **使用 HTTPS**：避免密钥与对话内容明文传输
3. **设置合理超时**：流式请求建议客户端超时 ≥ 120s
4. **处理流式断连**：客户端应能处理网络中断，必要时重试非幂等场景需谨慎
5. **模型名称**：使用平台已接入的模型标识，与模型管理中的名称一致

---

10. 常见问题

Q：能否使用 OpenAI 官方 SDK 直接调用?

A：请求路径、鉴权头、请求体结构兼容 OpenAI Chat Completions。需将 SDK 的 `base_url` 指向 `{host}/api/openai/v1`，并将 `api_key` 替换为平台 LLMKey。

Q：为什么没有返回 `llmKey`？

A：常见原因：租户未开通大模型接口访问、当前登录用户无 LLMKey、或用户/租户权限不足。请联系管理员检查租户配置。

---

11. 变更记录

版本	日期	变更内容
v1.0	2026-06-11	初稿：Chat Completions 对外接口、LLMKey 鉴权、错误码与并发说明
v1.1	2026-06-11	文档版本号更新至 v1.1
v1.2	2026-06-30	补充 `messages[].content` 数组格式（Vision）及图片 Base64 校验说明；补充 `n`、`stop`、`presence_penalty`、`frequency_penalty`、`user` 等接收但不转发的字段；更新错误码表与错误示例
v1.3	2026-07-20	补齐 Chat Completions 请求参数并改为完整透传；新增消息工具调用、结构化输出、推理、流选项、音频等参数说明与示例；`stop` 支持字符串或数组
v1.4	2026-07-21	补充非流式与流式响应参数说明，包括 `choices`、`message`、`delta`、工具调用、推理字段、Token 用量明细及 `finish_reason`
v1.5	2026-07-21	按参考接口 Schema 补充请求与响应中的嵌套对象字段，包括工具函数结构、流式用量选项、响应格式、音频配置及响应对象层级